CYBERMIND AI · TEAM GUIDE



Understand it, demo it.

What CYBERMIND AI is, how it works, what it contains, and how to run it live, in plain English and technical terms. For the whole team.

FOR THE PROJECT TEAM

EDITION 1 · OCTOBER 2026

CONTENTS

What is inside

PART I · UNDERSTAND IT

1 What CYBERMIND AI is 3

2 How it is built 5

PART II · HOW IT WORKS

3 Protecting the login 7

4 Protecting the session 9

PART III · WHAT IT CONTAINS

5 The defender console 11

6 VaultApp, the protected app 12

7 The Analyst Assistant 13

PART IV · RUN IT

8 Run it and demo it 15

PART V · TEAM REFERENCE

9 Cheat sheet and viva Q&A 17

1

What CYBERMIND AI is

The problem, the idea, and the two sides of the product, in plain English first and technical terms second.

1.1 The problem

Most online accounts are not hacked by genius hackers. They are taken over in two boring ways:

1. **Credential stuffing.** Criminals buy lists of leaked email and password pairs and try them on other sites, thousands per minute, from thousands of different IP addresses so no single address looks suspicious.
2. **Session hijacking.** Someone steals a valid logged-in session (a token), and now the website thinks *they* are the customer.

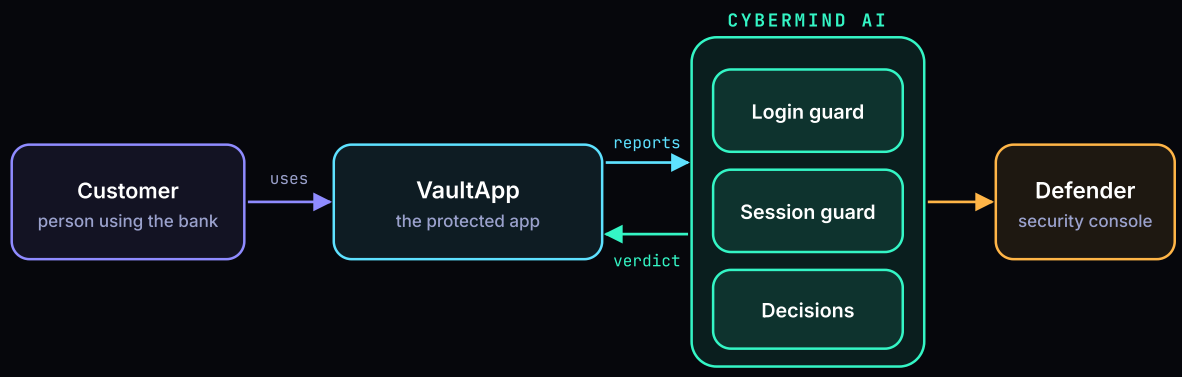
A normal login page stops neither: the passwords are real, and the stolen session is real. What is wrong is the **behaviour**.

> IN PLAIN ENGLISH
>
> CYBERMIND AI is a security guard that stands behind the front door of an app. It does not only check *who has the key*. It also watches *how the person walks*: how they type, how fast they click, which device and network they use. If a stranger is wearing the owner's key but walks differently, the guard stops them.

1.2 The idea in one sentence

> CYBERMIND AI detects account takeover from **behaviour and campaigns**, not just passwords and IP addresses, explains every decision, and lets the app enforce it.

1.3 The two sides of the product



The bank enforces the verdict. CYBERMIND decides; the customer never sees CYBERMIND.

Two sides. The customer uses the bank (VaultApp). The defender uses the console. CYBERMIND sits in between.

SIDE	WHO	WHAT THEY SEE	WHERE
Customer side	A person banking	VaultApp: sign in, balance, transfers. Never sees CYBERMIND.	`/#user-portal`
Defender side	The security / IT team	The console: live events, campaigns, sessions, simulations	`/` then "Sign in to console"
The engine	Automatic	Login guard, session guard, decision maker	`backend/app/`

1.4 What it does, in four verbs

- **Notice.** Groups failed logins into *campaigns* by client fingerprint (not by IP), and learns each user's normal typing and device.
- **Decide.** Fixed rules + a per-user baseline + a machine-learning model add up to a verdict: **allow**, **step-up** or **block**.
- **Explain.** Each verdict carries human-readable reasons ("typing 6 standard deviations from this user's normal").
- **Contain.** The app obeys: pause sensitive actions (HTTP 428) or freeze the session (HTTP 423). Only a human can unlock a freeze.

> REMEMBER
>
> CYBERMIND is a **behavioural** detector with **explanations**. It is not an antivirus, not a firewall, and not a replacement for MFA. Chapters 5, 6 and the Q&A list exactly what it cannot do.

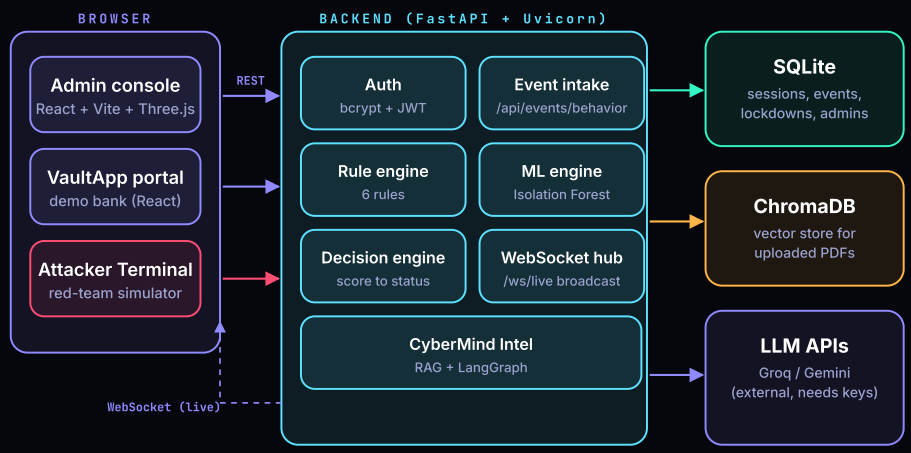
2

CHAPTER 2

How it is built

Components, technology choices and where everything lives in the code.

2.1 The architecture



Browser apps talk to one FastAPI backend over REST; the live feed uses a WebSocket. Data lives in SQLite (security data) and ChromaDB (documents for the assistant).

The system has three groups: **browser apps** (React), **one backend** (FastAPI, Python) and **storage** (SQLite, ChromaDB). The browser apps are the defender console, the VaultApp demo bank, and the Attack Simulator inside the console.

> IN PLAIN ENGLISH
>
> Think of a bank branch. The browser apps are the counter and the manager's office. The backend is the back office where decisions are made. The database is the filing cabinet. The WebSocket is the intercom that lets the manager's screen update the instant something happens.

2.2 Technology and why

TECHNOLOGY	ROLE	WHY WE USE IT
FastAPI + Uvicorn	Web API server	Fast, automatic validation and docs at `/docs`
Pydantic	Request and response shapes	Rejects malformed input before it reaches our code
SQLAlchemy + SQLite	Database layer	Zero setup; swap to Postgres with one setting
bcrypt (passlib)	Password hashing	Slow on purpose, so stolen hashes are hard to crack

TECHNOLOGY	ROLE	WHY WE USE IT
JWT (python-jose)	Admin login token, with revocation	Stateless sessions; logout revokes the token id
HMAC session tokens	Proof a browser owns a session	Knowing a session id is not enough to send events
scikit-learn Isolation Forest	Anomaly detection	Learns "normal" without needing labelled attacks
WebSockets	Live console feed	Events appear instantly, no refresh
React 18 + Vite + Three.js	Console, VaultApp, 3D scenes	Component UI and the neural-network journey
ChromaDB + LLM APIs	Analyst Assistant (optional)	Ask questions about uploaded security reports

2.3 Folder map

```
threat-detection-system/
  backend/app/
    main.py              starts the app, wires the routers, CORS, startup checks
    config.py            all settings (.env) + production safety checks
    security.py          IP resolution, rate limiters, signed session tokens
    auth.py              bcrypt, JWT, admin check, ingest auth, app keys
    rule_engine.py       the fixed rules and their weights
    baseline.py          per-user "normal" (running mean and spread)
    ml_engine.py         Isolation Forest wrapper
    decision_engine.py   points → tier (allow / step-up / block) + reasons
    campaign_engine.py   login campaigns, decoys, breached-password signals
    lab.py               the Realistic Attack Lab
    routers/             HTTP endpoints (events, bank, campaigns, logs, lab ...)
    models.py            database tables
  backend/tests/         43 automated tests
  frontend/src/          App.jsx, components/ (console screens, UserPortal = VaultApp)
  demo/                  attack_login.py, session_hijack.py (live demos)
```

IN OUR PROJECT

The most important files to read first are `decision_engine.py` (how points become a verdict), `campaign_engine.py` (login protection) and `routers/bank_routes.py` (how an app obeys CYBERMIND).

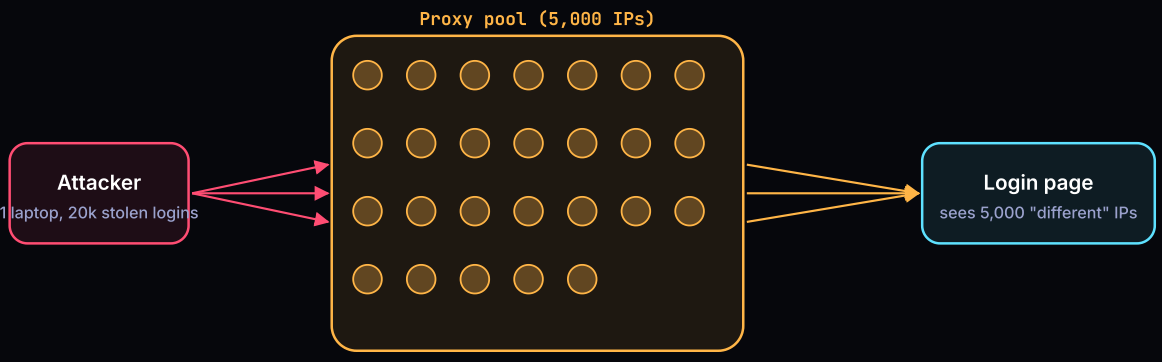
3

CHAPTER 3

Protecting the login

How CYBERMIND catches credential stuffing even when every attempt comes from a different IP.

3.1 Why per-IP rules fail

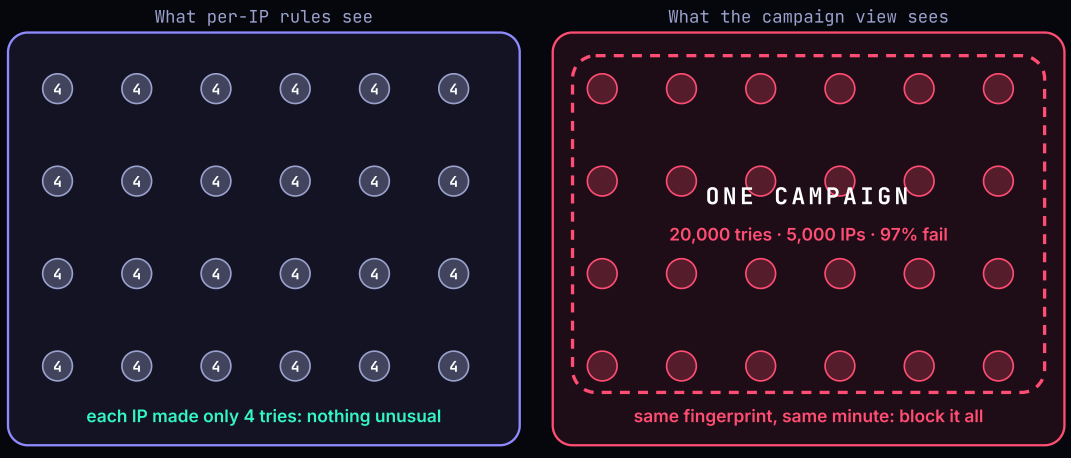


To the server it looks like thousands of different people. The attacker is one.

One attacker, a proxy pool of thousands of IPs: the login page sees thousands of different people.

A classic defence is "block an IP after 5 failures". An attacker with 5,000 IPs makes **one** try per address. Nothing trips. The attack is invisible *per IP* but obvious *as a whole*.

3.2 Our answer: campaigns



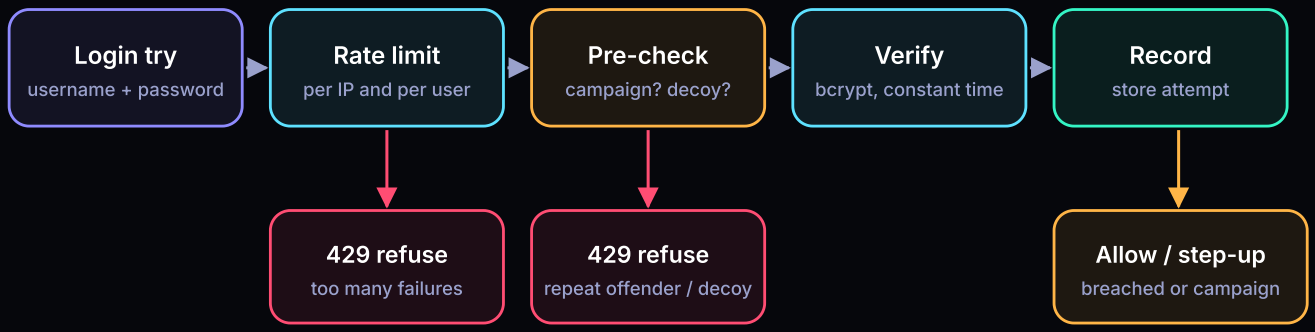
Left: what per-IP rules see. Right: what the campaign view sees: one tool, one fingerprint, one minute.

Every login attempt is stored with a **client fingerprint**: the browser family plus the *order of the HTTP headers* the client sends (real browsers and scripts differ here, and a script cannot cheaply change it). Attempts with the same fingerprint inside a 5 minute window form a **campaign** and are scored:

SIGNAL	POINTS	MEANING
Many attempts (100+)	2	Volume from one fingerprint
80% or more failures	3	Normal sites see a few percent
Failures from 60+ different IPs	3	IP rotation
Many usernames, ~1 try each	2	Stuffing or spraying
20%+ breached passwords	2	Known leaked passwords in use
A decoy account touched	**6**	No real person ever uses it

At **3 points** a campaign is *watched*; at **6** it is *blocked*.

3.3 What a login goes through



A blocked campaign marks every sign-in from that client fingerprint as step-up; known-good returning users are not punished.

The login path in VaultApp (the admin console login uses the same steps).

1. **Rate limit** failures per IP and per username.
2. **Pre-check** before touching the password: is this client part of a blocked campaign, or did it touch a decoy?
3. **Verify** the password with bcrypt. When the username does not exist we still do the same work, so response time never reveals which usernames are real.
4. **Record** the attempt for future scoring.

> IN PLAIN ENGLISH
>
> **Decoy accounts** are fake accounts like `svc_backup` that nobody legitimate uses. If anyone tries to log in as one, they are an attacker, with almost no chance of a false alarm. It is a tripwire.

3.4 What "blocked" really does

A blocked campaign does **not** lock everyone with a common browser fingerprint out. Repeat offenders (3+ failures from the same IP) and decoy touchers are refused outright (HTTP 429). Everybody else from that fingerprint gets **step-up**: a correct password still signs in, but the session cannot move money until verified. Customers with a good history are not punished.

HONEST LIMIT

A human-like bot that copies a real browser fingerprint and stays slow can hide in normal traffic. The Attack Lab includes this case and reports it as **missed**.

PART II · HOW IT WORKS

CHAPTER 4

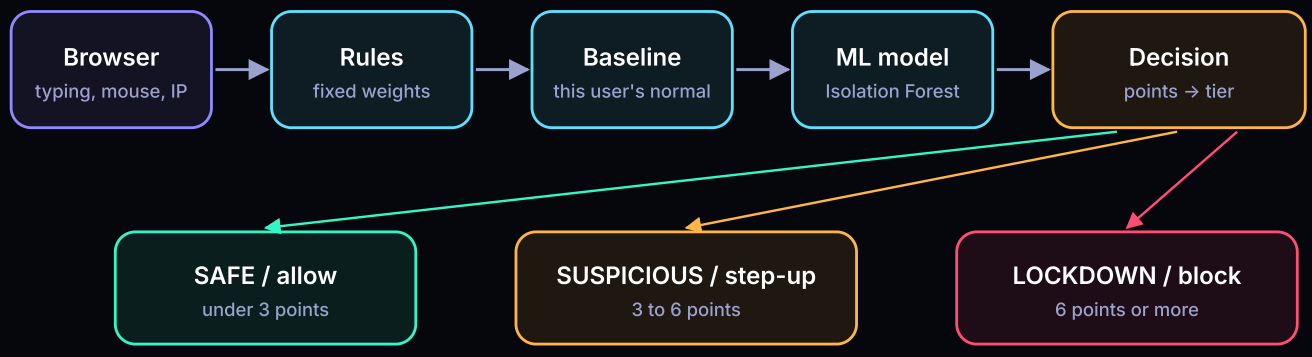
4 Protecting the session

How CYBERMIND judges a signed-in user from how they behave, then turns that into allow, step-up or block.

4.1 What the browser reports

After sign-in, VaultApp sends a small report every 2 seconds: typing speed, gap between keystrokes and how much it varies, click rate, mouse-movement variety, session length, IP address and device. **What you type is never sent**, only the rhythm.

4.2 Three judges, one verdict



Every verdict is stored with its reasons in plain English. The ML model on its own adds only 2.5 points.

Each report passes the rules, the personal baseline and the ML model; their points add up to a tier.

JUDGE	QUESTION IT ASKS	POINTS
Rules	Is something plainly wrong? (typing faster than 600 chars/min, request bursts, many failed logins, IP change)	Fixed weights, e.g. scripted typing 4.0
Baseline	Is this far from *this user's own* normal? (running mean and spread per user, after 20 clean events)	Grows with the size of the deviation
ML model	Is this odd compared with everyone? (Isolation Forest, starts from a starter dataset, then retrains on real traffic once there is enough)	2.5

Tiers: under 3 points = **SAFE** (allow); 3 to 6 = **SUSPICIOUS** (step-up); 6 or more = **LOCKDOWN** (block).

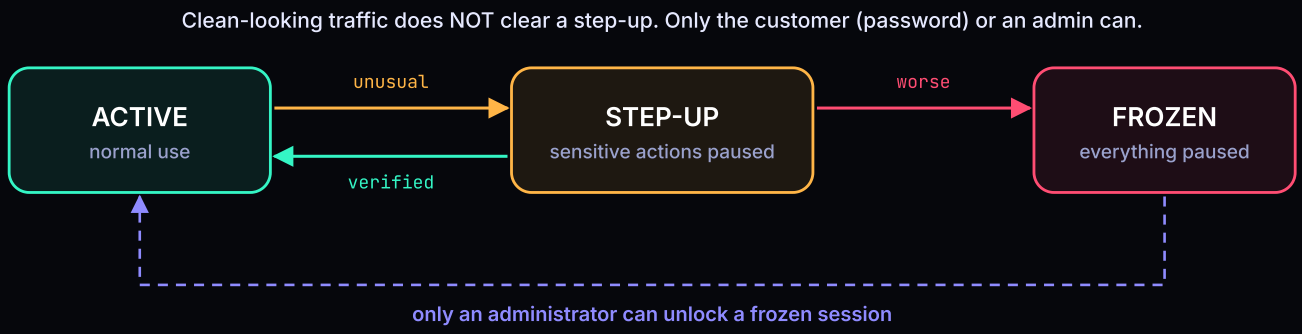
IN PLAIN ENGLISH

Imagine a nightclub doorman. Rules are the dress code. The baseline is knowing that *this* regular always arrives alone and orders the same drink. The ML model is the gut feeling that something is off. One alone may not stop you; together they decide.

4.3 Isolation Forest, simply

The model builds many random decision trees that try to isolate each data point. **Odd points are isolated in very few splits**; normal points take many. Short path = anomaly. It needs no labelled attacks, which is why it suits a project without real attack data. It is also why it only gets 2.5 points: on its own it is not trusted to freeze anyone.

4.4 The three states



A step-up is sticky: normal-looking traffic does not clear it.

IN OUR PROJECT

We found and fixed a real flaw while building this: originally one clean-looking report cleared a pending step-up within seconds, so an intruder only had to behave for one tick. It is now sticky, with a regression test.

4.5 Every decision is explained

Each stored event holds its tier and its reasons, for example *"+4.0 Typing speed beyond human range"* or *"+2.0 Session now comes from a different public IP"*. Clients only learn the qualitative outcome, never the numbers, so attackers cannot tune their behaviour against a score. The log exports to CSV or JSON Lines for a SIEM.

CHAPTER 5

5 The defender console

Every screen, what it is for, and when you would use it.

The console opens with a scroll-through-the-neural-network journey. Each feature is a **station**: scroll (or press keys **1 to 6**), click the glowing node, and the feature assembles on screen. **Esc** goes back. **Ctrl+K** opens quick actions. The sidebar groups the screens by what a defender does:

GROUP	SCREEN	WHAT IT IS FOR
MONITOR	**Overview**	Live threat status, event counts, lockdown count, activity chart, and a *hardening status* card (which security settings are on)
MONITOR	**Detections**	Every decision with flags, tier and reasons. Filter by status, search, live tail, export CSV or SIEM (.jsonl). Second tab: **Campaigns & Decoys**
RESPOND	**Active Sessions**	All monitored sessions and their state. **Unlock** a frozen session; **Clear step-up** once someone is verified
VALIDATE	**Attack Simulator**	*Manual Techniques* (spawn a sandbox target, identity churn, request flood, test a protected action) and the **Realistic Attack Lab**
SETUP	**Integrations**	How a company plugs CYBERMIND into its app: the two hooks, the app key, decoy count
ASSIST	**Analyst Assistant**	Upload security reports (PDF) and ask questions; answers come from your documents (needs a Groq or Gemini key)

5.1 Detections: Campaigns & Decoys

Shows each login campaign (BLOCKED / watching), its scale (attempts, IPs, usernames), its signals and **why it was flagged** in plain English, with a **Release** button for a false alarm. Below are the **decoy accounts** (add or remove), and the most recent login attempts with their fingerprint and decision.

5.2 The Realistic Attack Lab

Replays scripted attacks, mixed with hundreds of ordinary users, through the **real detection code** on a throw-away in-memory database (your data is never touched). Per scenario it reports: was it caught, at which attempt, how many accounts fell first, and **how many normal users were wrongly denied**.

SCENARIO	EXPECTED
Control: normal users only	nothing flagged
Naive burst, rotating IPs, password spraying, decoy touch, low and slow	caught
Human-like bot copying a browser fingerprint	caught if volume is high

SCENARIO	EXPECTED
Session thief with a different typing rhythm	caught
Diluted stealth, session thief copying the owner perfectly	**missed (by design)**

> **WHY SHOW MISSES?**
>
> A security tool that claims it catches everything is lying. The lab deliberately includes two cases it cannot catch, so your result is honest. That honesty is a strength in a viva.

5.3 Security of the tool itself

Admin endpoints and the live feed need the admin token; logout revokes it; events need a signed session token; CORS is an allowlist; client IP headers are trusted only from known proxies (dev switch aside); logins and ingestion are rate-limited; in `APP_ENV=production` a weak secret or password stops the app from starting.

PART III · WHAT IT CONTAINS

CHAPTER 6

6 VaultApp, the protected app

A real demo bank that shows how any company would plug CYBERMIND in.

6.1 Why it exists

A detector needs something to protect. **VaultApp** is a small bank with real accounts. It is deliberately built like a customer's own product: it owns its customers and passwords, reports to CYBERMIND, and **obeys the answer**.

6.2 What a customer can do

- **Open an account:** username, name, password (8+ characters; passwords found in known breaches are refused). Starts with a demo balance of $12,480.60 and five sample transactions.
- **Sign in:** passes through the login guard from Chapter 3.
- **Accounts, Transfer, Security, Profile:** a real transfer (review, confirm, balance checks); a *Security* page that shows the customer what CYBERMIND measures.
- **Money is fake.** Nothing real moves.

6.3 How the bank obeys CYBERMIND

SITUATION	WHAT THE BANK DOES	HTTP
Normal behaviour	Transfer goes through	200

SITUATION	WHAT THE BANK DOES	HTTP
Suspicious session	Banner "verify it is you"; transfers need the password again (`/bank/step-up/verify`)	428
Frozen session	Full-screen freeze; only an admin can unlock	423
Breached password at sign-in	Signs in, but with step-up already required	200 + flag

```
Browser  --sign in-->  /bank/login  --(campaign, decoy, breach checks)-->  session created
Browser  --every 2s-->  /user/behavior  (X-Session-Token)  -->  verdict stored, console updated
Browser  --transfer-->  /bank/transfer  -->  423 frozen | 428 step-up | 200 sent
```

IN PLAIN ENGLISH

CYBERMIND is the alarm system; VaultApp is the bank that decides whether to open the vault when the alarm says "unsure". The alarm cannot open or close the vault itself; the bank enforces the verdict.

6.4 Honest limits of the demo bank

KNOW THESE BEFORE THE VIVA

Step-up is the password again, not a real second factor. An attacker who already has the password could pass it. In a real deployment, step-up is an MFA code, an email link or a call-back; the hook (`/bank/step-up/verify`) is where that plugs in. Sessions and rate limits live in memory, so a restart resets them. The VaultApp IP-change demo needs the development switch `ALLOW_CLIENT_IP_HEADER=true` .

PART III · WHAT IT CONTAINS

CHAPTER 7

7 The Analyst Assistant

A separate helper that answers questions about security reports you upload. It does not detect attacks.

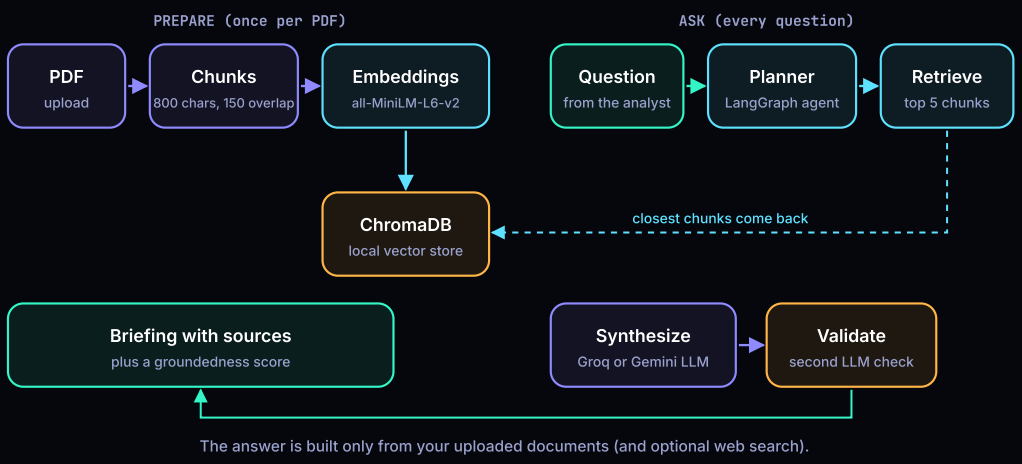
7.1 What it is

The **Analyst Assistant** (earlier called CyberMind Intel) is an AI helper for the *human* analyst. You upload a PDF (an incident report, a threat bulletin, a playbook) and ask questions in plain language. It answers **from your documents** and shows its sources. It lives in its own database, so it can never change detection data, sessions or logs.

IN PLAIN ENGLISH

Imagine a very fast intern who has read every report you handed them. You ask "what did this report say about stolen sessions?" and they point to the exact pages. They are told not to guess; if the report does not say, they say so.

7.2 How it works: RAG



RAG (retrieval-augmented generation): find the relevant passages first, then let the language model write an answer from only those passages.

1. **Prepare (once per PDF).** The PDF is split into **chunks** of about 800 characters with 150 characters of overlap, so no sentence is cut off from its context. Each chunk is turned into an **embedding**, a list of numbers that captures its meaning (model `all-MiniLM-L6-v2`, runs locally). The vectors are stored in **ChromaDB** on disk.
2. **Ask.** A small **LangGraph** agent plans the steps. If documents exist, it first **retrieves** the 5 chunks whose meaning is closest to your question (similarity search).
3. **Synthesize.** A language model (Groq or Gemini) writes a structured threat-intelligence briefing using only the retrieved text: summary, key findings, technical analysis, threat assessment, ATT&CK mapping if cited, evidence, takeaway.
4. **Validate.** A second model pass checks whether each claim is supported by the source and returns a **groundedness score (1 to 5)** and a hallucination flag.

PART	TECHNOLOGY	FILE
PDF reading and chunking	LangChain text splitters	`cybermind/pdf_loader.py`
Embeddings + vector store	sentence-transformers + ChromaDB	`cybermind/vector_store.py`
Agent: plan, retrieve, synthesize, validate	LangGraph	`cybermind/workflow.py`
API (admin only)	FastAPI	`routers/cybermind_routes.py`

7.3 Using it

1. Put `GROQ_API_KEY` or `GEMINI_API_KEY` in `backend/.env` and restart the backend.

2. Console > **Analyst Assistant**: upload a PDF (max 20 MB), wait for the "Indexed" message.
3. Pick a model and ask. The page shows the plan, the briefing, the sources and the groundedness check. **Clear** wipes only this store.

7.4 Honest limits

KNOW THESE BEFORE THE VIVA

It needs internet and an API key. Without them the question fails with a clear message; the rest of CYBERMIND works fine offline. **It can still be wrong.** RAG reduces made-up answers but does not remove them, and answer quality depends on the PDF text (scanned images are not read). **The checker is soft.** If the validation call itself fails, the code falls back to a default "verified" result, so treat the groundedness score as a hint, not a proof. **Web search is optional and unfiltered.** If the planner picks it, snippets come from a public search engine.

PART IV · RUN IT

CHAPTER 8

8

Run it and demo it

Start the system, then five live demos (A to E) you can do in front of a teacher. All run on your own laptop.

8.1 Start everything (two terminals)

TERMINAL 1: BACKEND

```
cd threat-detection-system/backend
python -m uvicorn app.main:app --port 8000
```

Check: open `http://127.0.0.1:8000/health` and see `"status":"ok"`.

TERMINAL 2: FRONTEND

```
cd threat-detection-system/frontend
npm run dev
```

Open `http://localhost:5173`. The admin username and password are the `ADMIN_USERNAME` and `ADMIN_PASSWORD` in `backend/.env`.

Set up two browser windows side by side: Window A = the defender console (click *Sign in to console*). Window B = the customer (click *Try the live demo*). Keep the console on **Overview** or **Detections**.

8.2 Demo A: a normal customer (2 minutes)

1. In Window B choose **Open account**, create a customer, sign in.
2. Send a small transfer. It goes through (HTTP 200).
3. In Window A, **Active Sessions** now lists that customer as ACTIVE and **Detections** shows green SAFE events arriving live.

Say: "A real customer is never disturbed. CYBERMIND is invisible when nothing is wrong."

8.3 Demo B: credential stuffing from many IPs (3 minutes)

In a third terminal (backend running):

```
cd threat-detection-system
python demo/attack_login.py
```

It fires 300 wrong-password logins, **each from a different IP**, then lets the attacker "guess" a real password. Watch Window A: a red **campaign blocked** banner appears, and **Detections > Campaigns & Decoys** shows the campaign BLOCKED with its reasons. At the end the script prints that the stolen login worked but the transfer was refused with **HTTP 428**.

Say: "Each IP tried once, so per-IP rules saw nothing. We caught the whole attack by its fingerprint, and even a correct password could not move money."

8.4 Demo C: a stolen session (3 minutes)

```
python demo/session_hijack.py
```

The script signs a customer in, replays 35 calm events (the owner), then a "thief" reuses the stolen session from a new IP and browser, typing like a machine. The thief's session is **frozen** and the $9,000 transfer returns **HTTP 423**. In Window A open **Active Sessions**, find the customer, press **Unlock**.

Say: "The token was valid, the password was never needed. Behaviour gave the thief away."

8.5 Demo D: the Attack Lab (3 minutes)

Console > **Attack Simulator** > **Realistic Attack Lab** > choose **QUICK** > **Run the lab**. After about a minute you get a table: attacks caught, accounts compromised, **normal users denied (0)**, and two scenarios honestly marked **MISSED**.

Say: "This is not a staged video. It runs our real detection code and tells the truth, including what we miss."

8.6 Demo E: the Analyst Assistant (3 minutes, optional)

Needs a key in `backend/.env` (`GROQ_API_KEY` or `GEMINI_API_KEY`) and internet. Console > **Analyst Assistant** > upload any security report PDF > wait for "Indexed" > ask *"What attack technique does this report describe and how do we detect it?"*. Show the **sources** list and the **groundedness** check.

Say: "It only answers from the document we gave it, cites it, and a second pass checks the answer against the source."

8.7 Run the automated tests

```
cd threat-detection-system/backend
python -m unittest discover -s tests -t .
```

Expect **43 tests, OK.** They cover rules, campaigns, decoys, baselines, the bank flow, sticky step-up, rate limits and API security.

> IF SOMETHING DOES NOT WORK
>
> **"Invalid credentials" on the console:** use the exact admin values in `backend/.env`, with no stray spaces. **Demos print "NOT blocked" or cannot reach the server:** restart the backend (it does not auto-reload) and make sure it is on port 8000. **Frozen test customer:** unlock it in Active Sessions, or just create a new one.

PART V · TEAM REFERENCE

CHAPTER 9

9 Cheat sheet and viva Q&A

The one-page summary, key endpoints and settings, and the questions you are most likely to be asked.

9.1 The 30-second pitch

"CYBERMIND AI protects an app against account takeover. It catches credential-stuffing campaigns by client fingerprint instead of IP, and hijacked sessions by comparing behaviour to each user's own baseline. Rules, a per-user baseline and an Isolation Forest vote; the result is allow, step-up or block, with reasons. The app enforces it. We tested it with a lab that honestly reports what we miss."

9.2 Key endpoints

ENDPOINT	PURPOSE	AUTH
`POST /bank/register`, `/bank/login`	Customer sign-up and protected sign-in	none (protected by the guard)
`POST /user/behavior`	Behaviour telemetry for a session	session token
`POST /bank/transfer`	Move money; returns 200, 428 or 423	session token

ENDPOINT	PURPOSE	AUTH
`POST /bank/step-up/verify`	Customer re-enters password	session token
`POST /auth/login` , `/auth/logout`	Admin sign-in and revocation	none / admin
`GET /api/sessions` , `/api/logs` , `/api/campaigns`	Console data	admin token
`POST /api/lockdown/unlock` , `/api/sessions/{id}/step-up/clear`	Human releases	admin token
`POST /api/auth-events/login-attempt`	Any app reports a login attempt	`X-App-Key`
`POST /api/lab/run`	Start the Attack Lab	admin token
`POST /api/cybermind/ingest` , `/query` , `/clear`	Analyst Assistant: upload PDF, ask, wipe	admin token
`GET /health`	Status and number of config warnings	none

9.3 Key settings (`backend/.env`)

SETTING	MEANING
`APP_ENV`	`development` warns about weak config; `production` refuses to start
`JWT_SECRET` , `ADMIN_PASSWORD`	Must be strong in production
`CORS_ORIGINS`	Which websites may call the API
`ALLOW_CLIENT_IP_HEADER`	Dev only: believe a client-supplied IP (needed for the demo scripts)
`IP_CHANGE_POLICY`	`step_up` (default) or `lock`
`DECOY_USERNAMES`	Fake accounts that act as tripwires
`APP_API_KEYS`	Keys for apps that report logins
`LOCKDOWN_THRESHOLD`	Points for a freeze (default 6.0)
`GROQ_API_KEY` , `GEMINI_API_KEY`	Needed only for the Analyst Assistant

9.4 Likely viva questions

Q1. What is new compared with a normal login rate limit? It groups attempts by client fingerprint, so an attacker rotating thousands of IPs still looks like one campaign.

Q2. Why Isolation Forest? It needs no labelled attacks, it is fast, and it isolates rare points in few splits. We give it only 2.5 points because it alone is not trusted to freeze anyone.

Q3. How do you avoid false alarms? Per-user baselines (a fast typist is only odd *for others*), decoys that have zero legitimate users, step-up before freeze, and the lab measures normal users wrongly denied (zero).

Q4. What cannot it catch? A slow bot that copies a real browser, and a thief who copies the owner's behaviour and device perfectly. Both are in the lab as MISSED.

Q5. Is step-up real MFA? No. The demo uses password re-entry; the `/bank/step-up/verify` hook is where a real second factor plugs in.

Q6. Is this a real product or a simulation? The detection, bank, database, hashing and enforcement are real code running live. The *traffic* in the lab and scripts is simulated, and the bank's money is fake.

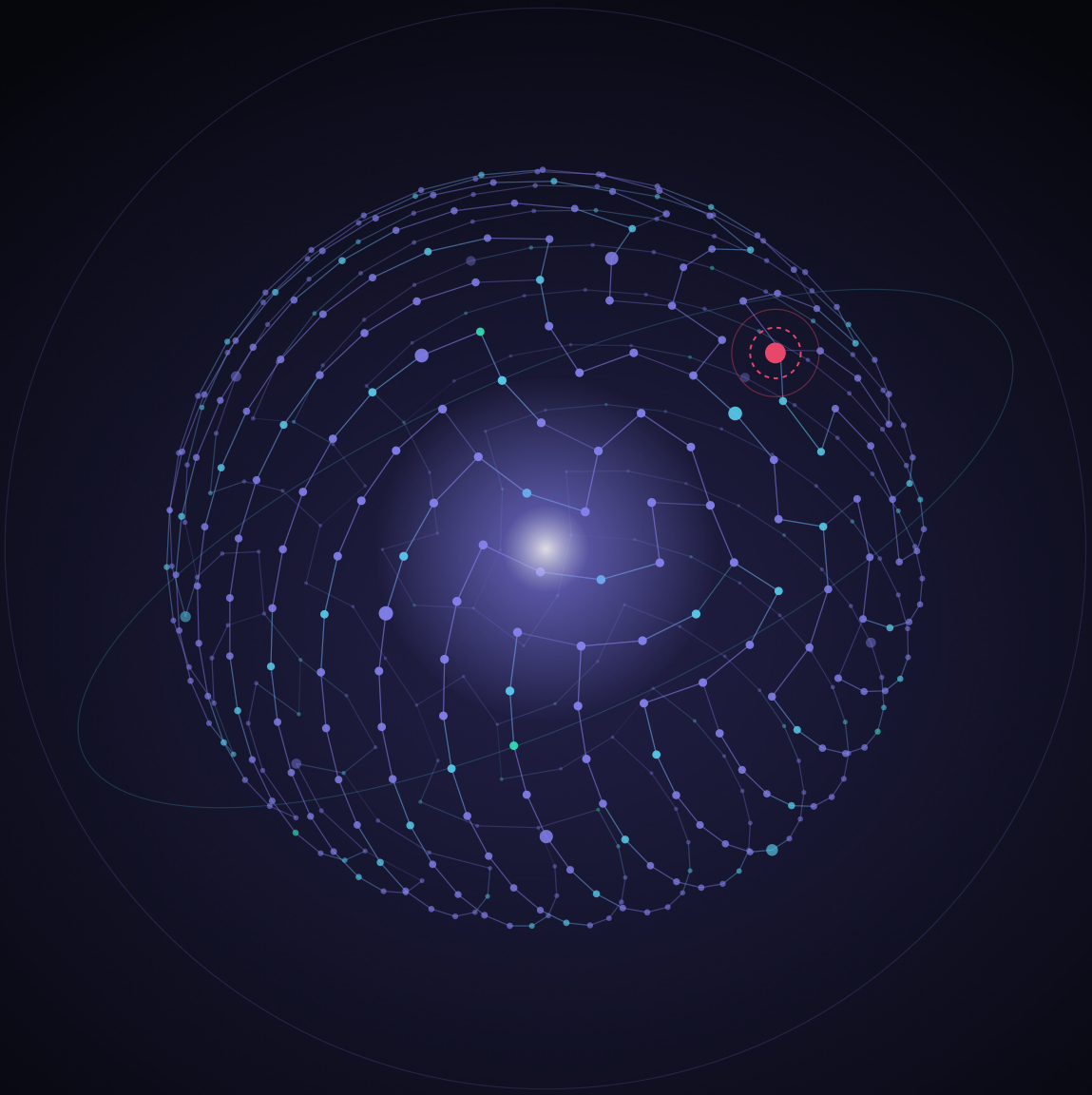
Q7. What happens if it is wrong? Step-up is a small nuisance (re-enter password); a freeze needs an admin to unlock. Every decision has reasons so the admin can judge.

Q8. Does the Analyst Assistant detect attacks? No. It is a separate helper for analysts: it answers questions about uploaded reports. Detection is done by the rules, baseline and ML model; the assistant never touches detection data.

Q9. How would you deploy it? Set `APP_ENV=production`, strong secrets, list trusted proxies, turn off the client-IP switch, use Postgres, plug a real MFA into step-up, and keep the model retraining on real traffic.

9.5 Words to know

Credential stuffing: trying leaked passwords on other sites. **Campaign:** many attempts that belong to one attacker. **Fingerprint:** traits of the client software, such as header order. **Decoy / honeytoken:** a fake account used as a tripwire. **Baseline:** a user's own normal. **Step-up:** extra verification before a sensitive action. **Lockdown:** a freeze only an admin can lift. **Anomaly:** a point that looks unlike the rest. **SIEM:** a company's central log system.



Notice. Decide. Explain. Contain.

CYBERMIND AI · TEAM GUIDE

EDITION 1 · OCTOBER 2026